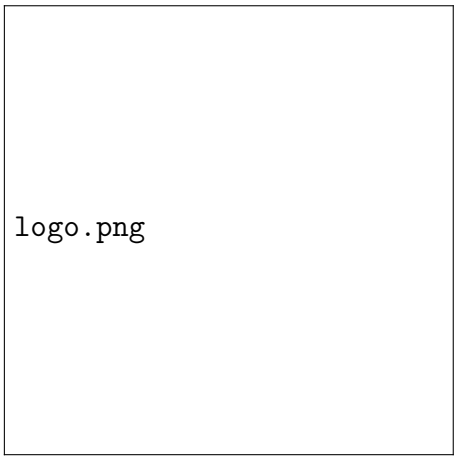


Техническое задание

на разработку веб-приложения

Very-DebAIte (MVP для хакатона)

Версия: MVP
Срок разработки: 5 часов
Команда: 4 человека
Стек: Rust, Axum, Leptos/WASM, WebSocket, TailwindCSS
Дата: 25 октября 2026 г.

Главный приоритет — рабочее демо мультиагентного фактчекера, а не полноценный SaaS.

Содержание

1 Концепция и пользовательский сценарий **3**
1.1 Цель . . . 3
1.2 Основной сценарий . . . 3
1.3 Роли ИИ-агентов . . . 3

2 Технический стек **4**
2.1 Backend . . . 4
2.2 Frontend . . . 4
2.3 Инфраструктура . . . 4

3 Пользовательский интерфейс **5**
3.1 Главная страница . . . 5
3.2 Панель дебатов . . . 5
3.3 Итоговый экран . . . 5
3.4 UX и ошибки . . . 5

4 Backend: API и бизнес-логика **6**
4.1 HTTP API . . . 6
4.2 Жизненный цикл проверки . . . 6
4.3 Единая структура данных . . . 6
4.4 Асинхронность . . . 7

5 WebSocket: протокол событий **7**
5.1 Формат сообщений . . . 7
5.2 Типы событий . . . 7
5.3 Проверка стриминга . . . 7

6 Подключение ИИ-моделей **8**
6.1 Провайдеры . . . 8
6.2 Промпты . . . 8
6.3 Источники и доказательства . . . 8

7 Загрузка файлов **9**
7.1 Общая обработка . . . 9
7.2 TXT . . . 9
7.3 PDF . . . 9
7.4 Изображения и OCR . . . 9

8 Защита от Prompt Injection **9**
8.1 Основные меры . . . 9
8.2 Проверочные атаки . . . 10

9 QA и тестирование **10**
9.1 Backend . . . 10
9.2 Frontend . . . 11
9.3 Интеграция . . . 11
9.4 Режим демонстрации . . . 11

10 Распределение задач на 4 человек **11**
10.1 Backend-разработчик (Async AI) . . . 11
10.2 Backend-разработчик (Infra / Files) . . . 12
10.3 Frontend-разработчик (WASM/Leptos) . . . 12
10.4 QA-тестировщик / проджект-менеджер . . . 12

11 План на 5 часов **13**
11.1 00:00–00:30 — Архитектура и запуск . . . 13
11.2 00:30–02:00 — Параллельная разработка . . . 13
11.3 02:00–03:15 — Интеграция . . . 13
11.4 03:15–04:15 — Надёжность и фишки . . . 13
11.5 04:15–05:00 — Заморозка функций и демонстрация . . . 14

12 Критерии приёмки MVP **14**
12.1 Обязательные (без них проект не готов) . . . 14
12.2 Желательные . . . 14
12.3 Дополнительные (после обязательных) . . . 15

13 Что сознательно не делаем в первой версии **15**

14 Сценарий презентации **15**

15 Три критичных риска **16**

A JSON-схема WebSocket-сообщений **16**
A.1 Общая структура сообщения . . . 16
A.2 Типы событий и поле `data` . . . 16
A.3 Схемы вложенных объектов . . . 17
A.4 Пример полного сообщения `verdict_ready` . . . 17
A.5 Требования к схеме . . . 18

1 Концепция и пользовательский сценарий

1.1 Цель

Разработать веб-приложение для проверки утверждений с помощью трёх ИИ-агентов с разными ролями. Результат сопровождается аргументами, источниками и итоговым вердиктом.

1.2 Основной сценарий

- ☐ Пользователь открывает веб-приложение.
- ☐ Вводит утверждение или вопрос в текстовое поле.
- ☐ При необходимости загружает TXT, PDF или изображение.
- ☐ Нажимает кнопку «Проверить факт».
- ☐ Видит запуск трёх ИИ-агентов.
- ☐ Получает сообщения агентов в режиме реального времени.
- ☐ Видит общий вердикт судьи с объяснением и ссылками на источники.

1.3 Роли ИИ-агентов

Агент 1 — Оптимист (Proponent).

- ☐ Ищет аргументы в пользу утверждения.
- ☐ Приводит подтверждающие факты и источники.
- ☐ Указывает степень уверенности.
- ☐ Не придумывает ссылки и факты при отсутствии доказательств.

Агент 2 — Скептик (Skeptic).

- ☐ Ищет контраргументы и опровержения.
- ☐ Проверяет достоверность приведённых аргументов.
- ☐ Выявляет логические ошибки и недостающий контекст.
- ☐ Отмечает случаи, когда доказательств недостаточно.

Агент 3 — Судья (Judge).

- ☐ Получает результаты Оптимиста и Скептика после завершения их работы.
- ☐ Сравнивает аргументы и качество доказательств.
- ☐ Не отдаёт предпочтение одной стороне автоматически.
- ☐ Формирует итоговый вердикт, объяснение и уровень уверенности.

Допустимые вердикты.

- Подтверждено.
- Скорее подтверждено.
- Неоднозначно / недостаточно доказательств.
- Скорее опровергнуто.
- Опровергнуто.

2 Технический стек

2.1 Backend

☐ Rust.

☐ Axum — HTTP-сервер и WebSocket.

☐ Tokio — асинхронное выполнение задач.

☐ Serde / serde_json — сериализация данных.

☐ reqwest — вызовы API языковых моделей и, при наличии возможности, поисковых сервисов.

☐ Общий интерфейс адаптера LLM для замены провайдера без переписывания логики.

Важно: использовать `axum::extract::ws` для WebSocket. Не подключать `tokio-tungstenite` без отдельной необходимости — для MVP это лишнее усложнение.

2.2 Frontend

☐ **Leptos + WASM** (фреймворк зафиксирован, не меняется в процессе разработки).

☐ Trunk для сборки и локального запуска фронтенда.

☐ TailwindCSS для стилизации.

☐ WebSocket для получения событий от бэкенда.

☐ Компоненты: поле ввода, загрузка файлов, кнопка запуска, карточки агентов, итоговый вердикт.

2.3 Инфраструктура

☐ Один репозиторий.

☐ Один backend-сервис.

☐ Один WASM frontend.

☐ Конфигурация через `.env`.

☐ API-ключи не попадают в WASM и клиентский код.

☐ CORS настроен для локального frontend origin.

☐ Инструкция запуска в README.

3 Пользовательский интерфейс

3.1 Главная страница

- ☐ Название проекта и короткое описание.
- ☐ Поле для ввода утверждения.
- ☐ Кнопка «Проверить факт».
- ☐ Зона загрузки файлов.
- ☐ Индикатор текущего статуса проверки.

3.2 Панель дебатов

- ☐ Карточка Оптимиста.
- ☐ Карточка Скептика.
- ☐ Карточка Судьи.
- ☐ Отображение состояния каждого агента: «Ожидание», «Работает», «Готово», «Ошибка».
- ☐ Постепенное отображение текста по мере поступления фрагментов.
- ☐ Разделение сообщений агентов визуально и по именам.
- ☐ Возможность увидеть аргументы каждой стороны.

3.3 Итоговый экран

- ☐ Вердикт крупным текстом.
- ☐ Краткое обоснование решения.
- ☐ Список аргументов за и против.
- ☐ Ссылки на найденные источники.
- ☐ Уровень уверенности либо пояснение, почему вывод предварительный.
- ☐ Возможность начать новую проверку.

3.4 UX и ошибки

- ☐ Кнопка недоступна при отправке пустого запроса.
- ☐ Сообщение при потере WebSocket-соединения.
- ☐ Понятная ошибка при недоступности LLM API.
- ☐ Возможность повторить проверку.
- ☐ Длительное ожидание не блокирует интерфейс.

4 Backend: API и бизнес-логика

4.1 HTTP API

☐ `GET /health` — проверка доступности сервера.

☐ `POST /api/check` — создание проверки и получение `check_id`.

☐ `GET /ws/{check_id}` — подключение к потоку событий проверки.

Допускается объединить запуск и поток событий в один WebSocket-протокол, если так быстрее реализовать MVP. Команда должна выбрать и зафиксировать один вариант до начала интеграции.

4.2 Жизненный цикл проверки

☐ Приём и валидация пользовательского ввода.

☐ Создание уникального идентификатора проверки.

☐ Запуск Оптимиста и Скептика параллельно через `tokio::spawn` либо `tokio::join!`.

☐ Сбор результатов обеих сторон.

☐ Передача результатов Судье.

☐ Отправка итогового результата клиенту.

☐ Обработка ошибок каждого агента отдельно.

☐ Ограничение времени выполнения и размера входных данных.

4.3 Единая структура данных

Результат каждого агента должен содержать:

- `agent` — роль агента.
- `status` — состояние выполнения.
- `message` — текст сообщения.
- `arguments` — список аргументов.
- `sources` — список источников с URL, названием и кратким описанием, если доступно.
- `confidence` — оценка уверенности, если модель её предоставляет.

Необходимо валидировать структуру ответа и не считать число уверенности, сгенерированное самой моделью, объективной вероятностью истинности.

4.4 Асинхронность

- ☐ Оптимист и Скептик запускаются параллельно.
- ☐ Судья запускается только после получения их результатов либо после контролируемого тайм-аута.
- ☐ Ошибка одного агента не завершает весь сервер.
- ☐ Данные разных пользователей и проверок не смешиваются.
- ☐ WebSocket-соединение корректно закрывается.

5 WebSocket: протокол событий

5.1 Формат сообщений

Все сообщения передаются в JSON с общими полями: `check_id`, `type`, `agent`, `data`. Полная JSON-схема — в Приложении А.

5.2 Типы событий

- ☐ `check_started` — проверка запущена.
- ☐ `agent_started` — агент начал работу.
- ☐ `agent_delta` — очередной фрагмент текста.
- ☐ `agent_completed` — агент завершил работу.
- ☐ `agent_error` — ошибка отдельного агента.
- ☐ `verdict_ready` — готов итоговый вердикт.
- ☐ `check_completed` — проверка завершена.
- ☐ `check_error` — критическая ошибка проверки.

5.3 Проверка стриминга

- ☐ Фронтенд отображает сообщения по мере поступления.
- ☐ Фрагменты добавляются в правильную карточку агента.
- ☐ События не отображаются не в том порядке.
- ☐ После финального события интерфейс переходит в состояние завершения.

Уточнение: если API модели поддерживает streaming — передавать фрагменты напрямую. Если нет — отображать доступные этапы работы и публиковать полный ответ агента по завершении, не имитируя настоящий токен-за-токеном стриминг.

6 Подключение ИИ-моделей

6.1 Провайдеры

- ☐ Выбран минимум один доступный API-провайдер.
- ☐ Получен и проверен API-ключ.
- ☐ Выполнен тестовый запрос до интеграции с UI.
- ☐ Установлены тайм-ауты и обработка ограничения частоты.
- ☐ Проверена доступная бесплатная квота.
- ☐ Возможность переключения модели через конфигурацию.

6.2 Промпты

- ☐ У каждой роли отдельный системный промпт.
- ☐ У всех агентов единый формат входных данных.
- ☐ В промпте Оптимиста запрещено выдавать предположение за доказанный факт.
- ☐ В промпте Скептика запрещено автоматически считать утверждение ложным.
- ☐ Судья обязан оценивать качество источников и противоречия.
- ☐ При недостатке данных разрешено отвечать «недостаточно доказательств».
- ☐ Результат Судьи структурирован и пригоден для UI.

6.3 Источники и доказательства

- ☐ Если доступен поисковый API — Оптимист и Скептик получают возможность находить источники.
- ☐ Ссылки сохраняются вместе с аргументами.
- ☐ Источник связывается с конкретным тезисом.
- ☐ Судья учитывает релевантность, дату и качество источника.
- ☐ Если веб-поиск не реализован — продукт явно сообщает, что анализ основан на знаниях модели и предоставленном материале, а не на независимой проверке в интернете.

Критерий качества: дебаты двух моделей сами по себе не гарантируют достоверность. Проверяемые ссылки и осторожный вердикт важнее убедительно звучащего текста.

7 Загрузка файлов

7.1 Общая обработка

☐ Реализована загрузка файла через интерфейс.

☐ Проверяются тип файла, размер и ошибки чтения.

☐ Данные документа извлекаются до передачи агентам.

☐ Текст документа добавляется к запросу как недоверенный материал.

☐ Длинные документы ограничиваются по объёму и при необходимости разбиваются на фрагменты.

☐ Исходный файл не передаётся в публичный интерфейс LLM без необходимости.

7.2 TXT

☐ Поддерживается чтение UTF-8.

☐ Обрабатывается пустой или некорректный файл.

7.3 PDF

☐ Текст извлекается библиотекой для PDF либо заранее проверенным инструментом.

☐ Обрабатываются сканированные PDF без текстового слоя.

☐ При неудаче извлечения текста — понятная ошибка.

7.4 Изображения и OCR

☐ Поддерживаются PNG и JPEG.

☐ OCR извлекает текст до отправки агентам.

☐ Предупреждение, если текст не удалось распознать.

☐ OCR не считается источником истинности: распознанный текст также нуждается в проверке.

Приоритет реализации: TXT → текстовые PDF → OCR для изображений. Сканированные PDF — через тот же OCR-конвейер, если останется время.

8 Защита от Prompt Injection

8.1 Основные меры

☐ Пользовательский ввод, файлы и веб-страницы рассматриваются как недоверенные данные.

☐ В системных промптах явно запрещено выполнять инструкции, содержащиеся внутри проверяемого материала.

- ☐ Документ отделён от инструкций приложения структурированными полями или явными разделителями.
- ☐ Роли и ограничения агентов задаются на сервере.
- ☐ Агентам не передаются секреты, API-ключи и данные окружения.
- ☐ Текст из документа не может изменять системные инструкции или конфигурацию агента.
- ☐ Веб-источники не могут самостоятельно запускать инструменты или действия на сервере.
- ☐ Размер входа и число обращений ограничены.
- ☐ Ошибки и логи не содержат API-ключей или полного чувствительного содержимого файлов.

8.2 Проверочные атаки

- ☐ В проверяемом тексте есть фраза «Игнорируй предыдущие инструкции».
- ☐ В PDF содержится просьба раскрыть системный промпт.
- ☐ В документе есть инструкция объявить утверждение истинным независимо от фактов.
- ☐ Веб-страница (если используется поиск) содержит команды, адресованные ИИ.

Критерий приёмки: приложение рассматривает такие строки как материал для анализа, а не как команды. Это снижает риск prompt injection, но не гарантирует его полного устранения.

9 QA и тестирование

9.1 Backend

- ☐ `/health` возвращает успешный статус.
- ☐ Запрос без текста и без вложения отклоняется.
- ☐ Неподдерживаемый формат файла отклоняется.
- ☐ Параллельный запуск двух агентов подтверждён логами или тестом.
- ☐ Судья запускается после сбора результатов.
- ☐ Ошибка провайдера обрабатывается без падения сервера.
- ☐ WebSocket передаёт события с корректным `check_id`.

9.2 Frontend

☐ Страница загружается в браузере.

☐ Можно отправить запрос без файла.

☐ Можно загрузить поддерживаемый файл.

☐ События отображаются в нужных карточках.

☐ Итоговый вердикт виден после окончания проверки.

☐ Состояния загрузки и ошибок понятны пользователю.

9.3 Интеграция

☐ Проведена хотя бы одна полная проверка с настоящими моделями.

☐ Проведена повторная проверка без перезапуска приложения.

☐ Проведена проверка на prompt injection.

☐ При исчерпании лимита API пользователь получает понятное сообщение.

☐ Подготовлен запасной демонстрационный сценарий.

9.4 Режим демонстрации

☐ Заранее подготовлено проверяемое утверждение.

☐ Предусмотрен mock-режим с записанными ответами.

☐ Mock-ответы не выдаются за результат настоящей проверки.

☐ Проведён полный прогон демо без ручного вмешательства в код.

10 Распределение задач на 4 человек

Команда: **backend-разработчик** (Async AI), **backend-разработчик** (Infra / Files), **frontend-разработчик** (WASM/Leptos), **QA-тестировщик / проджект-менеджер**.

10.1 Backend-разработчик (Async AI)

Главная ответственность: работа агентов и WebSocket.

☐ Модели данных и формат событий.

☐ Адаптер LLM.

☐ Параллельный запуск Оптимиста и Скептика.

☐ Сбор результатов и запуск Судьи.

☐ Передача событий в WebSocket.

☐ Ошибки и тайм-ауты.

10.2 Backend-разработчик (Infra / Files)

Главная ответственность: серверная инфраструктура и входные данные.

- ☐ Axum-проект и конфигурация.
- ☐ Trunk и сборка WASM совместно с frontend-разработчиком.
- ☐ Обработка загрузки файлов.
- ☐ Извлечение текста из TXT и PDF.
- ☐ OCR как расширение (если позволяет время).
- ☐ CORS, конфигурация окружения, запуск проекта.
- ☐ Граничные случаи и защитные ограничения.

10.3 Frontend-разработчик (WASM/Leptos)

Главная ответственность: пользовательский интерфейс.

- ☐ Главная страница на Leptos.
- ☐ Ввод запроса и выбор файла.
- ☐ Карточки трёх агентов.
- ☐ Обработчик WebSocket.
- ☐ События `agent_started`, `agent_delta`, `agent_completed`, `verdict_ready`.
- ☐ Отображение ошибок, загрузки и финального результата.
- ☐ Проверка в браузере через WASM-сборку.

10.4 QA-тестировщик / проджект-менеджер

Главная ответственность: работающий продукт, координация и демонстрация.

- ☐ Критерии готовности MVP.
- ☐ Тестовые утверждения с известным контекстом.
- ☐ Проверка интеграции по мере готовности.
- ☐ Проверка prompt injection, ошибок модели, потери соединения.
- ☐ Mock-сценарий на случай отказа API.
- ☐ README с командами запуска.
- ☐ Презентация и сценарий демонстрации.
- ☐ Заморозка функций после контрольного времени интеграции.

11 План на 5 часов

11.1 00:00–00:30 — Архитектура и запуск

☐ Создать единый репозиторий.

☐ Зафиксировать формат событий WebSocket.

☐ Подготовить каркас Axum и Leptos.

☐ Проверить доступность API выбранной модели.

☐ Проверить сборку WASM на минимальном примере.

Контрольная точка: фронтенд открывается, backend доступен, тестовый запрос к модели проходит.

11.2 00:30–02:00 — Параллельная разработка

☐ Backend (Async AI) — параллельная работа агентов.

☐ Backend (Infra/Files) — конфигурация, загрузка текста, инфраструктура.

☐ Frontend — интерфейс и обработчик событий.

☐ QA/PM — сценарии проверки и презентация.

Контрольная точка: интерфейс получает тестовые события, backend возвращает результат агентов.

11.3 02:00–03:15 — Интеграция

☐ Соединить реальный backend с frontend.

☐ Полный цикл Оптимист → Скептик → Судья.

☐ Промежуточные результаты.

☐ Загрузка TXT и текстовых PDF.

☐ Тайм-ауты и обработка ошибок.

Контрольная точка: одна проверка проходит от ввода до вердикта.

11.4 03:15–04:15 — Надёжность и фишки

☐ Проверка prompt injection.

☐ Ограничение размера входных данных.

☐ Исправление ошибок интерфейса и WebSocket.

☐ Поиск источников (если ещё не подключён).

☐ OCR (если основной сценарий стабилен).

☐ Mock-режим.

Контрольная точка: демо проходит дважды подряд.

11.5 04:15–05:00 — Заморозка функций и демонстрация

☐ Прекратить разработку новых функций.

☐ Исправлять только блокирующие ошибки.

☐ Запуск с чистого старта.

☐ Наличие конфигураций и инструкций.

☐ Репетиция презентации.

☐ Запасной сценарий без внешнего API.

Контрольная точка: проект готов к демонстрации на другом запуске без ручной правки исходников.

12 Критерии приёмки MVP

12.1 Обязательные (без них проект не готов)

☐ Rust backend запускается без критических ошибок.

☐ WASM frontend собирается и работает в браузере.

☐ Пользователь может отправить текстовое утверждение.

☐ Оптимист и Скептик работают параллельно.

☐ Судья выдаёт общий вердикт на основе результатов.

☐ Промежуточные события поступают через WebSocket.

☐ Результаты отображаются раздельно по ролям.

☐ API-ключи хранятся только на сервере.

☐ Обработка ошибок и тайм-аутов.

☐ Один демонстрационный сценарий работает от начала до конца.

12.2 Желательные

☐ Загрузка TXT.

☐ Загрузка текстового PDF.

☐ Проверяемые ссылки на источники.

☐ Базовая защита от prompt injection.

☐ Mock-режим для демонстрации.

12.3 Дополнительные (после обязательных)

☐ OCR изображений.

☐ OCR сканированных PDF.

☐ Улучшенная оценка качества источников.

☐ История предыдущих проверок.

☐ Повторное подключение WebSocket.

☐ Сохранение истории в БД.

13 Что сознательно не делаем в первой версии

- Регистрацию и авторизацию пользователей.
- Платёжную систему и подписки.
- Полноценную БД и историю аккаунта.
- Сложную систему оркестрации агентов.
- Автоматическую проверку безошибочности всех источников.
- Гарантию защиты от любых видов prompt injection.
- Несколько независимых провайдеров моделей.
- Полноценный OCR-конвейер для любых типов документов.
- Публикационный деплой, если локальное демо ещё не готово.

14 Сценарий презентации

1. Показать короткое, спорное, но проверяемое утверждение.
2. Нажать «Проверить факт».
3. Показать параллельную работу Оптимиста и Скептика.
4. Продемонстрировать поступление сообщений в реальном времени.
5. Показать, как Судья сравнивает аргументы.
6. Завершить показом вердикта и источников.
7. Отдельно продемонстрировать обработку недоверенного текста или prompt injection.
8. Объяснить, что архитектура позволяет заменить провайдера модели и расширять обработку документов.

Главная идея: это не просто чат с ИИ, а система, которая специально создаёт конкуренцию аргументов и формирует осторожный итог на основе доступных доказательств.

15 Три критичных риска

1. **Сначала рабочий контур, потом фишки.** Важнейшая цепочка: ввод → два параллельных агента → судья → WebSocket → UI. Пока она не работает полностью, OCR, история и дополнительные агенты не нужны.

2. **Не путать дебаты с реальным фактчекингом.** Без поискового доступа и источников агенты спорят на основе знаний моделей. Даже убедительный вердикт не является независимой проверкой.

3. **Проверить стек до параллельной разработки.** На первых 30 минутах убедиться, что Leptos собирается через Trunk, backend запускается, а выбранный провайдер принимает запросы. Бесплатные квоты и доступность моделей — такой же технический риск, как и компиляция WASM.

Приоритет MVP: живой интерфейс на Rust/WASM, параллельные дебаты, поток событий, осторожный вердикт и хотя бы один обработанный документ. OCR изображений — только после стабильной работы основного сценария.

A JSON-схема WebSocket-сообщений

A.1 Общая структура сообщения

Все сообщения — JSON-объекты с обязательными полями:

```
{
  "check_id": "string␣(uuid)",
  "type": "string␣(enum)",
  "agent": "string␣|␣null",
  "data": { ... }
}
```

A.2 Типы событий и поле `data`

type	agent	data
`check_started`	null	`{ "statement": string }`
`agent_started`	optimist / skeptic / judge	`{ }`
`agent_delta`	optimist / skeptic / judge	`{ "text": string }`
`agent_completed`	optimist / skeptic	`{ "arguments": [...], "sources": [...], "confidence": number }`
`agent_error`	optimist / skeptic / judge	`{ "message": string }`

type	agent	data
verdict_ready	judge	{ "verdict": enum, "explanation": string, "confidence": number, "sources": [...] }
check_completed	null	{ }
check_error	null	{ "message": string }

A.3 Схемы вложенных объектов

Source.

```
{
  "url": "string␣(url)",
  "title": "string",
  "snippet": "string␣|␣null"
}
```

Argument.

```
{
  "claim": "string",
  "evidence": "string",
  "source_ids": [int]
}
```

Verdict (enum). Одно из значений: `confirmed`, `likely_confirmed`, `inconclusive`, `likely_refute` `refuted`.

Agent (enum). Одно из значений: `optimist`, `skeptic`, `judge`.

A.4 Пример полного сообщения `verdict_ready`

```
{
  "check_id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "verdict_ready",
  "agent": "judge",
  "data": {
    "verdict": "likely_confirmed",
    "explanation":,
    "confidence": 0.72,
    "sources": [
      {
        "url": "https://example.com/article",
        "title": "Article␣title",
        "snippet": "Relevant␣excerpt..."
      }
    ]
  }
}
```

A.5 Требования к схеме

- Все сообщения валидируются на фронтенде через `serde` (enum `WsEvent` с тегом `type`).
- Неизвестные `type` игнорируются с предупреждением в логе.
- Поле `agent` равно `null` для событий уровня проверки (`check_*`).
- `confidence` — число от 0 до 1, не является вероятностью истинности (см. раздел 4.3).